

Activity 2.1 — Algorithm Design Report

UFCFYR-15-2 Advanced Algorithms | 2025/26

Ishaq Modassir Mushtaq

Activity 1.1 — Degree Calculation

Architecture: Module, DegreeResult, Student

The system was redesigned into three model classes in `models.py`. **Module** is a frozen dataclass — immutable after creation — that parses the module code string at construction via `Module.from_code()`, extracting credits and year level directly from the hyphen-delimited segments. **DegreeResult** encapsulates both averaging methods: `weighted_average()` for Level 6 and `level_5_best_100_average()` for Level 5, keeping them cohesive and independently testable. **Student** delegates all calculation to a `DegreeResult` instance, following the single-responsibility principle.

Key Algorithm — Best-100-Credit Selection

Level 5 modules are sorted by mark descending in $O(m \log m)$ time. Credits are accumulated greedily until the 100-credit cap is reached; partial credits are handled at the boundary so the total is always exactly 100. This is provably optimal: it is equivalent to the fractional knapsack problem (items with equal weight per credit), which the greedy approach solves exactly. The weighted average and the `level_5_modules_used` dictionary are returned together in a single pass.

Performance Optimisations

- `itertuples()` instead of `iterrows()` — over 100× faster on the 300-student DataFrame.
- Incremental `DictWriter` writes — no in-memory row accumulation.
- Frozen dataclass instances — smaller memory footprint than equivalent mutable classes.
- Module-name dict built once outside the student loop — $O(1)$ per lookup.

Justification

Separating `Module`, `DegreeResult`, and `Student` makes the 3:1 weighting rule and 100-credit selection explicit and auditable — both live in one place rather than scattered across the main script. Using a frozen dataclass for `Module` prevents bugs arising from accidentally mutating mark values after parsing.

Activity 1.2 — Password Generator

Algorithm — Recursive Backtracking with Pruning

The generator builds passwords character by character recursively, tracking four counters (`cap`, `low`, `num`, `spcl`) at every node. Two independent pruning conditions fire before recursion:

1. **Hard-limit prune:** if `cap > 2` or `spcl > 2`, abandon the branch immediately.

2. **Forward-looking prune:** if the number of unsatisfied required categories exceeds remaining positions (`missing > remaining`), the branch can never produce a valid password regardless of future choices.

Pseudocode

```
generate(list, length, cap, low, num, spcl):
    if cap > 2 or spcl > 2: return
    remaining = target - length
    missing = (cap==0)+(low==0)+(num==0)+(spcl==0)
    if missing > remaining: return
    if length == target:
        if cap>=1 and low>=1 and num>=1 and spcl>=1:
            add to buffer
        return
    chars = valid_starts if length==0 else all_chars
    for char in chars:
        d_c,d_l,d_n,d_s = char_deltas[char]
        append char
        generate(list, length+1, cap+d_c, low+d_l,
                num+d_n, spcl+d_s)
    pop char
```

char_deltas Precomputation

Before recursion begins, a `char_deltas` dictionary maps every character to its pre-computed (`cap_delta`, `low_delta`, `num_delta`, `spcl_delta`) tuple. This eliminates four set-membership tests per character placement, replacing them with a single $O(1)$ dict read — a meaningful saving at depths where millions of calls occur.

I/O Optimisation

A list-based write buffer of 10 000 entries is maintained and flushed to disk as a batch. The current length is passed as an explicit integer parameter rather than calling `len(password_list)` on every call.

Justification

The forward-looking prune eliminates entire subtrees structurally incapable of satisfying all four category constraints, particularly effective near the end of short passwords. Combined with precomputed deltas, both CPU and I/O overhead are reduced substantially over an unoptimised backtracker.

Activity 1.3 — Railway Route Planning

Architecture

Five modules: `graph_model.py` (directed graph & CSV loading), `dijkstra_alg.py` (shortest-path & TSP solver), `string_matcher.py` (fuzzy search), `route_visualizer.py` (PNG diagram), and `task1.3_main.py` (interactive CLI). Separating visualisation into its own class keeps the algorithm classes free of display concerns.

Dijkstra's Algorithm — $O((V + E) \log V)$

A min-heap priority queue (`heapq`) always expands the lowest-cost unvisited node. Because the railway graph is directed with asymmetric edge weights ($A \rightarrow B \neq$

$B \rightarrow A$), two separate Dijkstra calls are made for each ordered pair; symmetry is never assumed. Paths are reconstructed by propagating the path list through heap entries.

TSP Variant — Distance Matrix + Permutations

All pairwise shortest paths between required stations are precomputed into a distance matrix — each ordered pair computed exactly once — before TSP enumeration begins. The TSP is then solved by evaluating all permutations of intermediate stations over the cached matrix. Both path-TSP (start $\neq$ end) and cycle-TSP (start = end) variants are handled. For up to 7 intermediate stations this yields at most $7! = 5040$ permutations, which is exact and fast in practice.

Fuzzy Station Search

A custom similarity scorer combines: exact-match detection, substring containment with length penalty, sequential character matching (Levenshtein-like), and word-overlap boosting. The top-10 candidates are ranked and presented interactively, allowing users to confirm or retry without needing exact station names.

Route Visualiser — RouteVisualizer

After a route is computed the user may optionally export a PNG diagram. `RouteVisualizer.save_route_png()` uses **matplotlib** to position nodes in a serpentine grid layout, colour-coded by role (start, end, intermediate stop, transit). Directed edges are drawn with **FancyArrowPatch** arrowheads and labelled with the edge weight from the graph. The figure is saved at 220 DPI with a tight bounding box.

Justification

Precomputing the distance matrix avoids $O(n^2)$ redundant Dijkstra calls during TSP enumeration. The directed-graph model correctly reflects real rail pricing where outbound and return fares differ. The visualiser provides output the user can inspect without parsing a text file, targeting the 70–84% band criterion.

Activity 1.4 — Parallel Face Recognition

Parallelism Model — ProcessPoolExecutor

Python's GIL prevents true CPU-level parallelism with threads. The solution uses `concurrent.futures.ProcessPoolExecutor` to spawn one worker process per CPU core, distributing the image workload across all available cores. Time complexity becomes approximately $O(n/p)$ where n is the image count and p the core count.

Per-Image Optimisations

Each worker applies three stacked optimisations:

1. **No upsampling:** `upsample_num_times=0` skips expensive image rescaling, trading marginal accuracy for speed.
2. **Location pre-detection:** face locations are detected first, then passed directly into `face_encodings()`, preventing the detection CNN from running twice on

the same image.

3. **Small landmark model:** `model="small"` uses the 5-point landmark model instead of the default 68-point model — significantly faster at negligible accuracy cost for identity matching.

Early Termination

Results are consumed via `as_completed()`, which yields each future the moment it finishes rather than waiting for all. On the first confirmed match, all remaining pending futures are cancelled. Since exactly one match exists in the dataset, this can save close to $(n-1)/n$ of total compute time in the best case.

Justification

`ProcessPoolExecutor` is required over `ThreadPoolExecutor` because `face_recognition/dlib` is CPU-bound; only separate processes bypass the GIL. Workload distribution is automatic. Early termination is the critical efficiency mechanism: without it, the process pool continues scanning every remaining image even after a match is found.

Libraries and Unoriginal Work

Library / Tool	Purpose
pandas	CSV reading and DataFrame cleaning (Task 1.1)
dataclasses (stdlib)	Frozen Module dataclass for immutable module records (Task 1.1)
heapq (stdlib)	Priority queue for Dijkstra's algorithm (Task 1.3)
itertools.permutations	TSP enumeration (Task 1.3)
matplotlib	Route diagram PNG generation via RouteVisualizer (Task 1.3)
face_recognition	Face detection and encoding (Task 1.4) — by Adam Geitgey
cv2 (OpenCV)	Displaying matched image with bounding box (Task 1.4)
concurrent.futures	ProcessPoolExecutor for parallel processing (Task 1.4)
dlib	Backend for face_recognition ; pre-compiled wheel used on Windows

All core algorithm logic (backtracking, greedy credit selection, Dijkstra's implementation, similarity scoring, parallel task distribution) was written independently. The **face_recognition** and **dlib** libraries are third-party; their usage is acknowledged above.

References

- Cormen, T.H., Leiserson, C.E., Rivest, R.L. and Stein, C. (2022) *Introduction to algorithms*. 4th edn. Cambridge, MA: MIT Press.
- Dijkstra, E.W. (1959) 'A note on two problems in connexion with graphs', *Numerische Mathematik*, 1(1), pp. 269–271. Available at: <https://doi.org/10.1007/BF01386390> (Accessed: 20 March 2026).
- Geitgey, A. (2017) *face_recognition* [Python library]. Available at: https://github.com/ageitgey/face_recognition (Accessed: 15 March 2026).
- Held, M. and Karp, R.M. (1962) 'A dynamic programming approach to sequencing problems', *Journal of the Society for Industrial and Applied Mathematics*, 10(1), pp. 196–210. Available at: <https://doi.org/10.1137/0110015> (Accessed: 18 March 2026).
- Hunter, J.D. (2007) 'Matplotlib: A 2D graphics environment', *Computing in Science & Engineering*, 9(3), pp. 90–95. Available at: <https://doi.org/10.1109/MCSE.2007.55> (Accessed: 22 March 2026).
- King, D. (2009) *Dlib-ml: a machine learning toolkit*. Available at: <http://dlib.net> (Accessed: 15 March 2026).
- McKinney, W. (2022) *Python for data analysis*. 3rd edn. Sebastopol, CA: O'Reilly Media.
- Python Software Foundation (2024) *concurrent.futures* — *Launching parallel tasks*. Available at: <https://docs.python.org/3/library/concurrent.futures.html> (Accessed: 12 March 2026).
- Python Software Foundation (2024) *heapq* — *Heap queue algorithm*. Available at: <https://docs.python.org/3/library/heapq.html> (Accessed: 14 March 2026).
- Python Software Foundation (2024) *itertools* — *Functions creating iterators for efficient looping*. Available at: <https://docs.python.org/3/library/itertools.html> (Accessed: 14 March 2026).
- Sedgewick, R. and Wayne, K. (2011) *Algorithms*. 4th edn. Upper Saddle River, NJ: Addison-Wesley.
- University of the West of England (2023) *Academic Regulations 2023/24*. Bristol: UWE Bristol. Available at: <https://www.uwe.ac.uk/study/academic-information/regulations> (Accessed: 10 March 2026).
- Zhang, K. and Shasha, D. (1989) 'Simple fast algorithms for the editing distance between trees and related problems', *SIAM Journal on Computing*, 18(6), pp. 1245–1262. Available at: <https://doi.org/10.1137/0218082> (Accessed: 19 March 2026).